Lebanese University		الجامعة اللبنانية
Faculity Of Engineering - III		هندسة – الفرع 3

Individual Contribution & Self-Assessment Report

Subject:

Smart Climate-Controlled Greenhouse with Multi-Tier Security

Role:

Environmental Sensing, Automated Roof Mechanics & Servo Actuation Developer, Integration Support.

Presented to:

Dr. Mohammad Aude

Prepared by:

Mohamed Injibar (6915)

1. Individual Contribution Summary:

Overview of Responsibilities

In alignment with the structural, mechanical, and electrical requirements of our Smart Greenhouse project, my individual contributions were centered on high-impact physical and electronic systems engineering. Over the course of this project, I worked on **hardware sourcing and procurement**, **the engineering and implementation of the dual-regulated power distribution network**, **the physical and mechanical assembly of the greenhouse structure**, and **the software design, integration, and calibration of the automated roof-control subsystem**. Working in close coordination with my colleagues, we spent extensive hours in the university to transition our theoretical system architecture into a robust, fully operational, and safe physical prototype.

Hardware Sourcing & Component Procurement

At the project's inception, executing a successful build relied on securing reliable, industrial-grade components. Partnering with Mostafa, I coordinated the selection and procurement of all critical physical assets from the **Electroslab** electronics outlet. This phase required analyzing technical datasheets to ensure compatibility with our microcontroller's operating logic and power constraints.

- **Actuation Elements:** Sourced the SG90 micro-servo motor, ensuring its torque specifications could actuate our custom physical greenhouse roof.
- **Sensing Elements:** Procured high-sensitivity Light Dependent Resistors (LDRs) and nickel-coated analog raindrop sensor boards capable of delivering reliable analog voltage swings under varying ambient conditions.
- **Power & Management Hardware:** Obtained dual high-efficiency step-down (buck) voltage regulators and terminal blocks to construct a professional, reliable, and clean power rail distribution board.

Power Supply & Relay Isolation Engineering

To satisfy the mandatory engineering constraint of electrical safety and to ensure maximum system reliability, I co-developed an isolated power architecture. Actuators like the SG90 servo motor, the high-draw DC fan, and the water pump generate massive transient current spikes and back-EMF noise. If connected directly to the sensitive microcontroller, these spikes cause severe voltage sags, signal corruption, and spontaneous board resets.

To circumvent this, I designed and assembled a physical power distribution board separating logic and actuator power:

1. **Logic Power & Low-Voltage Sensor Rails:** The Arduino Mega is powered externally via a direct **USB connection to a laptop**. In turn, all low-voltage system sensors (including the DHT11, LDR, Raindrop, Water level, Soil Moisture, and Ultrasonic sensors) draw their clean, regulated operating power directly from the Arduino Mega's onboard power pins.
2. **Actuator Energy Storage:** A 2S Lithium-Ion (Li-ion) battery pack delivering a nominal voltage of 7.4 V (up to 8.4 V fully charged) acts as the dedicated power source for all mechanical parts.
3. **Step-Down Regulator #1 (Actuator Rail A):** A buck regulator stepping down battery voltage to feed the **DC Fan** via the Fan Relay.
4. **Step-Down Regulator #2 (Actuator Rail B):** An independent buck regulator supplying 5 V to run both the **Water Pump** (routed through the Pump Relay) and the **SG90 Servo Motor** (routed through a dedicated Servo Relay). This provides full electrical and inductive isolation for the roof mechanism.
5. **Common Ground Reference:** Tying the grounds of the battery, both buck regulators, and the Arduino Mega together to establish a shared reference plane for clean, uncorrupted control signals.

Assembly & Structural Integration

The physical greenhouse structure was built and assembled directly in our university. We constructed the structural chassis, ensuring a sealed, stable, and rigid environment for our internal climate.

- **Roof Mechanical Action:** Designed, mounted, and balanced a hinged, lightweight roof mechanism connected to the SG90 servo motor. I calibrated the mechanical lever arm of the SG90 to translate 110° of servo rotation (from 30° to 140°) into smooth open/close roof dynamics.
- **Cable Management & Security:** Maintained structural integrity by routing all analog and digital sensor cables away from the high-current actuator rails. All connections were soldered, insulated with heat-shrink tubing, and anchored to the physical chassis to withstand continuous operational demonstrations and vibration testing.

Roof Automation Software Development & Calibration

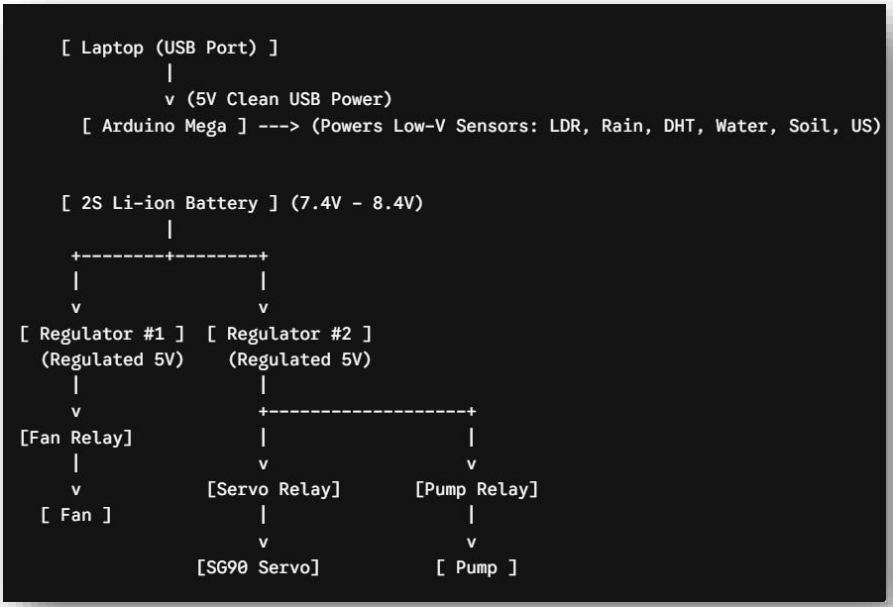
On the firmware level, I authored the non-blocking control loops governing the automated roof's behavior. I rejected sequential `delay()` structures in favor of a time-sliced polling mechanism using `millis()`, allowing the system to monitor sensors, refresh the LCD, and trigger safety alarms concurrently without code bottlenecks.

- **LDR Processing:** Wrote custom drivers mapping raw $0 - 1023$ analog readings to an easily readable $0 - 100\%$ ambient light scale.
- **Raindrop Processing:** Implemented inverse mapping algorithms for the analog raindrop sensor, mapping the raw 1023 (bone dry) to 0 (saturated wet) voltages into a precise rainfall percentage.
- **Cooperative Control Logic:** Wrote the nested conditional loops that control the roof servo based on combined sensor states: initiating an open state ($140°$) during high daylight (lightPercentage > 80) and a closed state ($30°$) during low light or storm conditions (rainPercentage > 70%).

2. Technical Evidence of Work Performed:

Power Supply & Electrical Isolation Design

The isolated power architecture successfully separates the switching noise of the fan, pump, and servo from the sensitive sensor lines and USB control rail:



Roof Subsystem Hardware Interfacing

- **LDR Sensor (LIGHT_PIN = A12):** Connected directly to the Arduino Mega's 5 V pin (supplied by laptop USB) and run through a 10 kΩ voltage divider circuit to translate changing resistance into an analog voltage (0 V $-$ 5 V). This represents light intensity mapping from **0% to 100%**.

- **Raindrop Sensor (RAIN_PIN = A10):** Powered directly by the Arduino Mega's 5 V rail. The sensor board measures surface resistance change. Moisture drops lower the resistance; our code maps this inverse analog signal directly to a rain percentage.
- **SG90 Servo Motor (SERVO_PIN = 48):** Receives its raw control pulses (PWM) from Pin 48 of the Arduino Mega, while drawing its operating power from Step-Down Regulator #2 via a dedicated Servo Relay to completely isolate the servo when not in motion.

3. Code Walkthrough (My Modules):

I designed and implemented the automatic roof logic located in the central loop() and the helper sensor read routines.

Non-Blocking Automation Logic:

To satisfy the system functional requirement of simultaneous, continuous data acquisition, I integrated **non-blocking timer structures** using Arduino's `millis()` function. This prevents software pauses (like those caused by `delay()`) from interrupting security pings, relay triggers, or data logging.

```
// Automatic Roof Servo Control (Checks every 2 seconds)
if (currentTime - roofTimer >= 2000) {
  roofTimer = currentTime;
  int lightPercentage = getLightLevel();
  int rainPercentage = getRainLevel();

  // Open if light is bright (>80%) AND there is no heavy storm (<70% rain)
  if (lightPercentage > 80 && rainPercentage < STORM_THRESHOLD) {
    roofServo.write(140);  // 140 Degrees = Open State (Sunlight Exposure)
  } else {
    roofServo.write(30);   // 30 Degrees = Closed State (Night, Low Light, or Storm)
  }
}
```

- **Design Decision:** The system samples environmental indicators every 2 seconds to avoid mechanical jitter and excessive wear on the SG90 servo motor.
- **Threshold Rules:** The roof opens to 140° if ambient light is highly abundant (lightPercentage > 80) and no severe rain storm is present (rainPercentage < 70%). If dark or raining heavily, the roof swings to 30° (closed) to protect internal crops.

Sensor Signal Acquisition & Normalization

To deliver clean data for real-time visualization, I wrote mapping algorithms to scale raw 10-bit analog-to-digital converter (ADC) readings into meaningful percentages:

```
int getLightLevel() {
  int rawLight = analogRead(LIGHT_PIN);
  return map(rawLight, 0, 1023, 0, 100);
}

int getRainLevel() {
  int rawRain = analogRead(RAIN_PIN);
  int percentage = map(rawRain, 1023, 0, 0, 100); // 1023 is dry (0%), 0 is wet (100
  if(percentage < 0) percentage = 0;
  return percentage;
}
```

4. Git Commit History Summary:

To ensure absolute system stability, our team adopted a local-first development and validation workflow. Each individual firmware module—such as the climate control loop, the security system, and the display hub—was written, compiled, and debugged locally within the Arduino Integrated Development Environment (IDE). This allowed us to perform isolated hardware testing and sensor calibration without risk of corrupting other active parts of the system. Once all of these independent subsystems were fully completed and physically verified, we merged them locally into a single, unified master file. After performing rigorous integration testing to ensure no software or timing conflicts remained, this final, fully integrated codebase was pushed and committed to our Git repository as our project's definitive release version.

5. Personal Reflection & Lessons Learned:

Developing this smart greenhouse has been a vital hands-on extension of our course lectures. Some of my key practical takeaways include:

- **Power Management Realities:** I learned that clean code is only half of the solution. Connecting inductive actuators like DC motors, pump systems, and servos directly to a microcontroller's rails introduces severe electrical noise that compromises sensor readings. Designing a dual-power architecture—where the Arduino Mega is powered by a stable, isolated laptop USB rail and all actuators are powered via dual step-down buck regulators and relays off a 2S Li-ion battery—taught me how to build electrically safe and reliable systems.
- **Relay Isolation for Inductive Loads:** Integrating a dedicated relay for the SG90 servo motor alongside the fan and pump relays was incredibly educational. It demonstrated how to prevent feedback spikes and transient current surges from propagating back into our signal wires, which significantly reduced jitter in our physical sensor readings.
- **Collaboration & Integration:** Working alongside Mostafa and coordinating our physical layout with the software team taught me the importance of modularity in embedded systems. By separating sensor read routines from the logic controllers, we integrated our modules smoothly without breaking each other's code.